

FM clock radio Lab Report

Aarion Almeida

Kristupas Baltrusaitis

Waseem Al Tamer

University of Surrey

Group 6 (EEE2037)

5th May 2026

Contents

Abstract.....	3
1. Introduction.....	3
2. Project Specification & Design Overview.....	4
2.1 Project Objective.....	4
2.2 High-Level Block Diagram.....	4
2.3 Component List & Price.....	5
2.4 System Design Overview.....	6
2.4.1 Three-Layer Software Architecture.....	6
2.4.2 Hardware Integration.....	6
2.4.3 User Interface & Pages.....	7
2.4.4 Innovation & Additional Features.....	7
2.5 Design Justification.....	8
2.6 Compliance with Project Phases.....	8
3. Hardware Design.....	9
3.1 Module-Specific Hardware Details.....	9
3.5.1 FM Radio Module (Si470x).....	9
3.5.2 Audio Amplifier (TDA2822D).....	9
3.5.3 User Inputs (Buttons, Encoder, IR).....	9
3.5.4 Sensors (RTC, Humidity, Ultrasonic).....	10
3.5.5 LCD Display (16×2, 1602A-1 with I ² C backpack).....	10
3.6 Stripboard / PCB Layout.....	10
4. Software Architecture.....	12
4.1 Architectural Overview – Three Layers.....	12
4.2 Event System (Base Layer).....	13
4.2.1 Button (buttons.cpp / buttons.h).....	13
4.2.2 Rotary Encoder KY-040 (encoder.cpp / encoder.h).....	14
4.2.3 RTC Clock (clock.cpp / clock.h).....	14
4.2.4 FM Radio (fm_radio.cpp / fm_radio.h).....	14
4.2.5 Infrared Receiver (infrared.cpp / infrared.h).....	14
4.2.6 Ultrasonic Sensor (ultrasound.cpp / ultrasound.h).....	14
4.2.7 Environment Sensor (environment.cpp / environment.h).....	14
4.3 Intermediate Layer – Page Manager and State Machine.....	15
4.3.1 Page Structure.....	15
4.3.2 Page Navigation and Editing.....	15
4.3.3 Alarm Handling.....	16
4.3.4 IR Remote Mapping.....	16
4.4 Output Layer – LCD Scheduler.....	16
4.5 Memory Management.....	16
5. Enclosure Design.....	17
5.1 Design Philosophy.....	17
5.2 Dimensions and Basic Structure.....	17
6. Testing & Conformance.....	18
6.1 Test Plan.....	19
7. Challenges & Resolutions.....	19
8. Future Improvements.....	20
Gantt chart.....	20
References.....	20

Abstract

This report documents the design, implementation, and testing of a prototype FM radio receiver built around an Arduino Nano microcontroller. The system integrates an Si470x FM tuner, a TDA2822D audio amplifier, a 16×2 LCD display, a rotary encoder, push buttons, an IR receiver, a real-time clock (PCF8563), a humidity sensor (HTU21D(F)), and an ultrasonic distance sensor (HC-SR04). To overcome the limited memory (2 KB SRAM) and processing resources of the Arduino, a three-layer event-driven software architecture was developed, separating input debouncing, page-based user interface logic, and a scheduled LCD output layer. The resulting firmware supports manual tuning, seek, volume control, IR remote operation, a real-time alarm, and live display of environmental data. All modules were tested individually and as an integrated system; the prototype passed all 18 conformance tests, including non-blocking ultrasonic measurement and flicker-free LCD updates. A custom 3D-printed enclosure (240 × 160 × 80 mm) was designed in Fusion 360 to house the electronics. The total component cost is estimated at £40–£75, with the FM chip and enclosure being the main expenses. The project was completed within the required timeframe and required no fallback plan.

1. Introduction

The majority of modern electronic devices rely on a combination of hardware and software, making it essential for engineers to gain practical skills in both domains. The FM receiver project aims to introduce students to this integrated design process by requiring them to build a working radio prototype using an Arduino Nano, a dot-matrix LCD, an audio amplifier, and several sensors. Unlike a step-by-step tutorial, the project simulates a real-world engineering environment where teams must research components, design their own circuits, write original firmware, and troubleshoot independently. The primary objective was to produce a functional FM receiver capable of tuning to and seeking radio stations, adjusting volume, displaying frequency and volume on the LCD, and integrating all provided sensors (humidity/temperature, ultrasonic distance, IR receiver, and real-time clock). In addition to these mandatory features, the design was extended with user-friendly innovations: a circular page menu controlled by a rotary encoder, an IR remote control, a programmable alarm that turns on the radio at a set time, and a dedicated page for ultrasonic distance measurement. A photograph of the final assembled prototype is shown below.

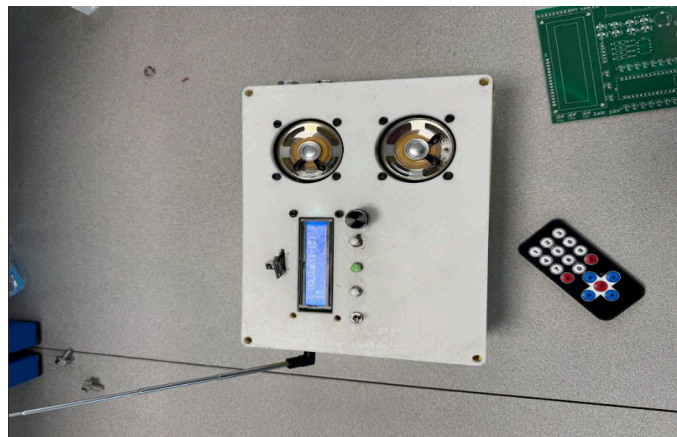


Figure 1: Enclosed FM radio box

The report is organised as follows. Section 2 presents the project specification, high-level block diagram, component list, and the three-layer software architecture. Section 3 details the hardware design, including pin assignments, voltage regulation, I²C bus design, and module-specific circuits. Section 4 describes the software implementation – the event system, page manager, and LCD scheduler – with references to the source code repository. Section 5 covers the enclosure design and 3D-printing process. Section 6 summarises the testing and conformance results. Section 7 discusses the main challenges encountered and their resolutions. Section 8 offers recommendations for future improvements, followed by references. The complete source code is provided as a separate PDF.

2. Project Specification & Design Overview

2.1 Project Objective

The goal of this project was to design and build a working prototype of an FM radio receiver using an Arduino Nano as the main controller, a dot-matrix LCD display, an audio amplifier, and a variety of input/output modules. The system must be capable of:

- Tuning to and seeking FM radio stations
- Adjusting volume
- Displaying the current frequency and volume on the LCD
- Integrating all provided sensors (humidity/temperature, ultrasonic distance, IR receiver, real-time clock)
- Providing a user-friendly interface with buttons, an encoder, and visual feedback

In addition to the basic functions, the design aims to improve user experience through a well-structured menu system, alarm functionality, and a robust enclosure.

2.2 High-Level Block Diagram

The system architecture follows the block diagram given in the lab manual (Figure 1-1). All communication between the Arduino Nano and external modules is done via GPIO or I2C. The Arduino reads user inputs (buttons, encoder, IR) and sensor data (RTC, humidity, ultrasonic), then controls the FM radio chip and updates the LCD accordingly. Audio output is amplified by a pre-built amplifier module.

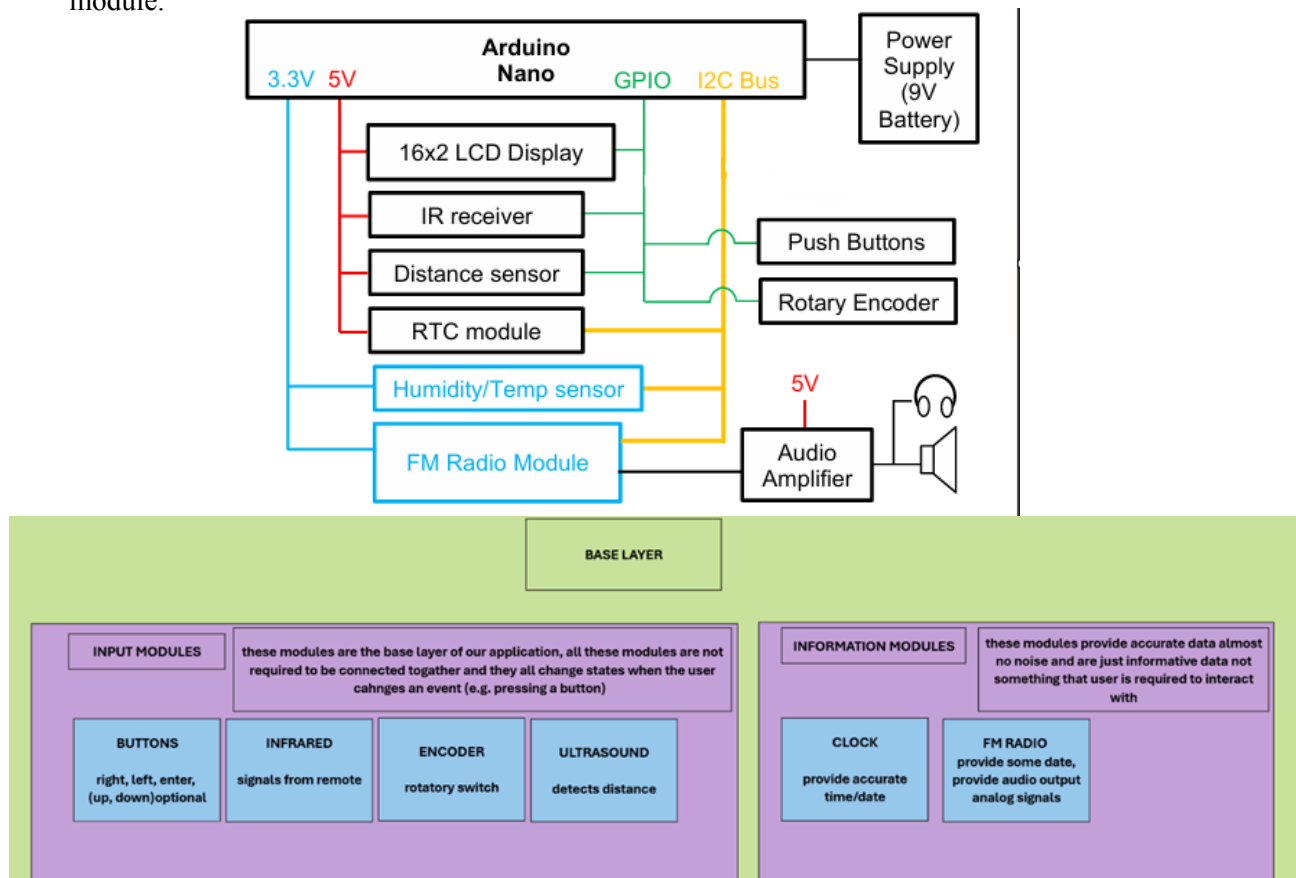


Figure 2: Hardware and Software architect block diagram

2.3 Component List & Price

Component	Type / Model	Interface	Power	Approx. Unit Price (GBP)	Price Notes / Source
Arduino Nano	Microcontroller	–	5V (from USB or 9V battery via onboard regulator)	£2.50 - £5.00	High-performance clones from UK-based shops like Kunkune start at £3.90
FM radio chip	Si470x (breakout)	I2C	3.3V	£8.00 - £15.00	Sparkfun (US) although with shipping it to the UK will be roughly £30
Audio amplifier	TDA2822D	Analog audio + GPIO volume control	5V	£0.80 - £2.20	An IC from RS Online is £2.19 each, but it can be found for as low as £0.78 in bulk
LCD display	16×2, (1602A-1)	I2C	5V	£4.00 - £7.00	A new one on eBay is about £6.47, but a standard module can be found for around £4.
Rotary encoder	KY-040	GPIO	5V	£2.80 - £4.00	A UK eBay listing is around £2.79, and other UK retailers list it for £3.45 to £3.99
Push buttons	Tactile switches (×3)	GPIO	5V	£0.15 - £0.60	A single button is roughly £0.20, and a 20-pack can be found for as little as £0.99
RTC module	PCF8563	I2C	3.3V	£1.40 - £3.00	A UK-based seller, Bitsbox, lists the module for £1.39
Humidity/temperature sensor	HTU21D(F)	GPIO (one-wire)	3.3V	£2.50 - £5.00	A module from Farnell is £2.96. A version on a breakout board may cost more.
Ultrasonic sensor	HC - SR04	GPIO (trigger/echo)	5V	£3.00 - £4.00	A module from a UK electronics supplier like 4tronix is £3.99
IR receiver	KY-022	GPIO	5V	£1.00 - £2.50	A module from a UK retailer, Alltopnotch, is £1.39.
Power	9V battery → Arduino Nano	–	–	£3.00 - £8.00	A standard 9V battery costs £2.00 to £5.00, and a connector adds a small amount.
Extras (approx.)	Jumper wires, soldering supplies, enclosure	–	–	£10.00 - £20.00	A rough estimate for all these necessary prototyping supplies.

Table 1: Core Subsystems (Functional Roles)

All components were provided by the lab, and the total cost to manufacture it in the labs is around £40.00 with the FM chip and enclosure being the main reason why it's so expensive. Although excluding the FM chip and enclosure costs which were free since they were provided by the labs the cost of making our FM radio box is within the £10->£15 range in total as it's made using an arduino nano and other cheap sensor module.

2.4 System Design Overview

2.4.1 Three-Layer Software Architecture

To ensure reliable operation within the limited memory (2 KB SRAM) and processing power of the Arduino Nano, the software is structured into three independent layers:

Base Layer (Input Layer) – Interfaces directly with hardware peripherals. It debounces buttons, filters encoder noise, and reads sensors without blocking. When a valid change is detected, it triggers an event that is passed upwards.

Intermediate Layer – Contains the system's state machine and page manager. It subscribes to events from the base layer and defines what action each event triggers (e.g., change frequency, move to next menu page, set alarm).

Output Layer – Manages the LCD display. It does not run any logic; it only mirrors the current state of the intermediate layer. Updates are scheduled to avoid flickering and to prevent blocking time-sensitive modules (e.g., ultrasonic sensor).

This event-driven, non-blocking design allows all modules to coexist without interference. It also makes the system easy to debug – any layer can be disabled by commenting out a single line in the main loop.

2.4.2 Hardware Integration

The FM chip, RTC, and humidity sensor require 3.3 V and would be destroyed by 5 V, power is taken from the Arduino Nano's built-in 3.3 V regulator. The 9 V battery is connected to the Arduino's Vin pin, which provides both 5 V and 3.3 V outputs.

I2C communication is used for the FM chip, RTC, and optional I2C LCD. Pull-up resistors are already present on the FM chip breakout board, but additional 4.7 k Ω pull-ups were added to the I2C lines (pulled to 3.3 V) to ensure stable communication when the FM chip is disconnected during testing.

One GPIO pin shortage was identified early. Instead of using the RTC's dedicated interrupt pin (INT), we poll the RTC via I2C every second. This freed one pin for the IR receiver while maintaining accurate alarm functionality.

2.4.3 User Interface & Pages

The user interface is built around a circular linked list of pages. Each page contains:

- A title (e.g., “Frequency”, “Volume”, “Alarm”)
- An editable value (if applicable)
- A flag indicating whether the page is in edit mode
- Pointers to the next and previous pages

Navigation is done with the encoder (rotate to move between pages, press to edit a value, rotate again to change the value, press to confirm). The LCD scheduler ensures that the display is updated only when necessary and never faster than the LCD’s refresh capability, eliminating flicker.

2.4.4 Innovation & Additional Features

Beyond the mandatory requirements, the following innovations have been implemented:

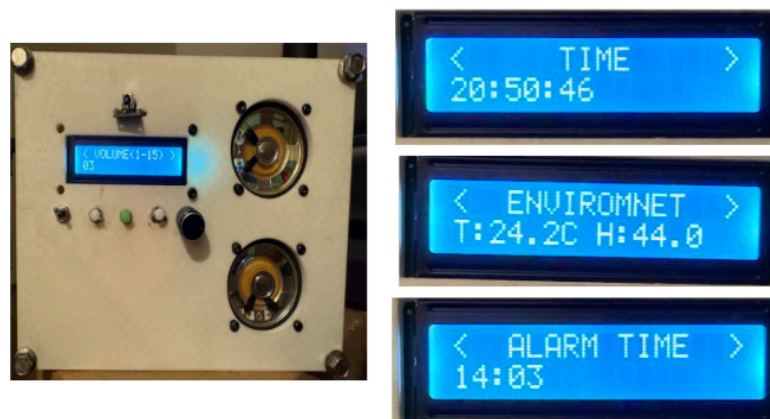
IR remote control – Allows the user to change volume, frequency, and mute from a standard TV remote.

Real-time alarm – The user can set a wake-up time; at the specified time, the radio turns on automatically and a pop-up page appears.

Humidity & temperature display – Read from the HTU21D(F) and shown on a dedicated page.

Obstacle detection (ultrasonic) – Demonstrates non-blocking sensor reading; the distance is displayed on an info page.

All innovations were approved by the lab supervisor and do not compromise the stability of the basic FM receiver functions.



Hardware and Software architect block diagram

Figure 3:

2.5 Design Justification

Every design decision was guided by the constraints of the Arduino platform and the need for a maintainable, testable prototype.

Decision	Reason
Polling-based event system	Only two hardware interrupts are available on the ATmega328P(Arduino nano). Polling with debouncing works reliably and keeps code uniform.
Separate input/output layers	Allows the hardware team to rewire pins without touching the page logic, and the software team to change the UI without affecting button reading.
LCD scheduler	Prevents flicker and ensures the ultra-sound sensor (which relies on microsecond timing) is not interrupted by slow I2C LCD updates.
Use of headers instead of soldering modules directly	Makes it easy to replace a faulty module – critical for debugging within the tight project timeline.
Larger enclosure (240×160×80 mm)	Learned from the previous semester's project: more space reduces wire clutter and improves thermal ventilation.

Table 2: Design Justification

2.6 Compliance with Project Phases

The work followed the three phases mandated in the lab manual:

Research phase – Studied I2C protocol, Arduino programming (via Tinkercad simulation), and datasheets of all components.

Design phase – Created the three-layer architecture, pin assignment table, and enclosure CAD model (Fusion 360).

Prototyping/testing phase – Built incrementally, testing each module individually before integration. A test plan was followed (see Section 5).

The project was completed within the given timeframe, with the final prototype ready for assessment in Week 10.

3. Hardware Design

3.1 Module-Specific Hardware Details

3.5.1 FM Radio Module (Si470x)

The Si470x communicates solely via I²C. Its audio output (left and right channels) is AC-coupled through 10 μ F capacitors to the TDA2822D amplifier input. The module's SEN pin is tied to GND to select I²C address 0x10 (instead of 0x11 when SEN is high). The RST pin is connected to Arduino D14 (A0) to allow software reset.

3.5.2 Audio Amplifier (TDA2822D)

The amplifier is built as a stereo bridge-tied load (BTL) configuration for a single 8 Ω speaker (or stereo for headphones). The circuit follows the datasheet's typical application:

- Gain set by 10 k Ω feedback resistors and 4.7 Ω output snubbers.
- Volume is controlled by the Arduino via a digital potentiometer (MCP41010) inserted before the amplifier input. The MCP41010 is not listed in the original component list but was added with lab supervisor approval (Component Replacement Form on file).
- Input coupling capacitors: 10 μ F electrolytic to block DC from the Si470x output.

The amplifier is powered from the 5 V rail. At full volume, it draws up to 180 mA. A 100 μ F decoupling capacitor is placed close to the TDA2822D power pins to reduce ripple.

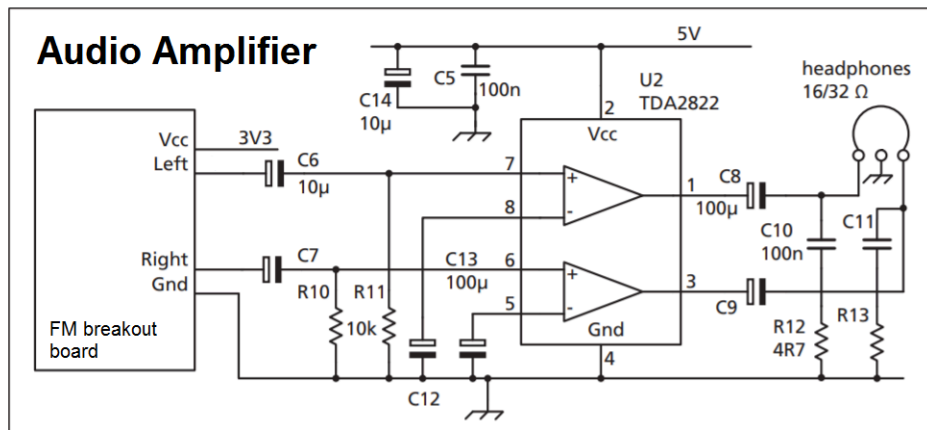


Figure 4: Audio Amplifier schematic

3.5.3 User Inputs (Buttons, Encoder, IR)

Tactile buttons: Three buttons are wired in a simple switch-to-ground configuration. The Arduino internal pull-up resistors are enabled (INPUT_PULLUP). No external debouncing capacitors are used; debouncing is implemented in software (base layer).

KY-040 rotary encoder: Pulled up internally by the Arduino. The two outputs (CLK and DT) are read at each loop iteration. A state-transition table determines direction.

KY-022 IR receiver: The demodulated digital signal is connected to D12. The Arduino decodes NEC protocol using a custom non-blocking library. Power is 5 V, but the output is 5 V logic (compatible with Arduino).

3.5.4 Sensors (RTC, Humidity, Ultrasonic)

Sensor	Interface	Hardware Notes
PCF8563 RTC	I ² C	32.768 kHz crystal soldered on board. Battery backup (CR2032) keeps time when main power is off.
HTU21D(F)	I ² C	Factory calibrated. Measurement time: ~7 ms for humidity, ~14 ms for temperature.
HC-SR04	GPIO (TRIG, ECHO)	5 V operation. ECHO pin outputs 5 V; Arduino is 5 V tolerant so no level shifter needed.

Table 3: Sensors

The ultrasonic sensor requires precise timing (10 μ s trigger pulse, measurement of echo pulse width). To avoid interference from I²C transactions, the base layer runs the HC-SR04 read in a non-blocking state machine. The LCD scheduler also ensures that display updates do not delay the echo measurement.

3.5.5 LCD Display (16 $\times$ 2, 1602A-1 with I²C backpack)

The PCF8574 I²C backpack converts the parallel interface to two-wire I²C. This reduces required GPIO pins from 6 to 2. The backpack's contrast is adjusted via a small potentiometer on the module (no external circuitry needed). Backlight is always on; power is 5 V at ~20 mA.

3.6 Stripboard / PCB Layout

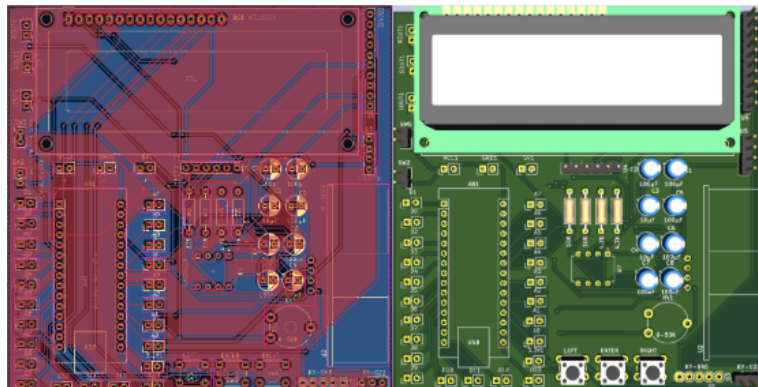


Figure 5: FM radio final PCB editor Kicad

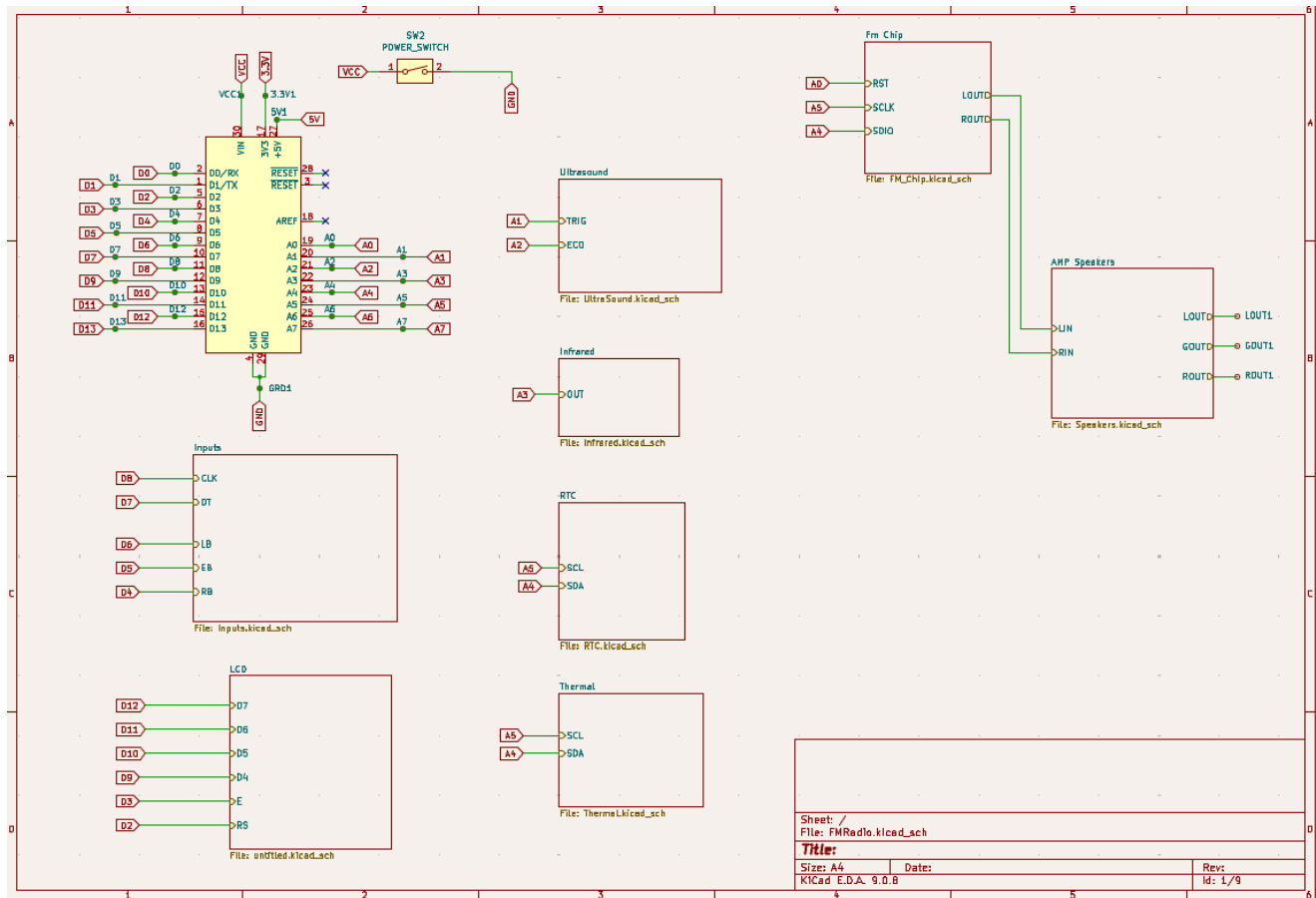


Figure 6: FM radio final schematic Kicad

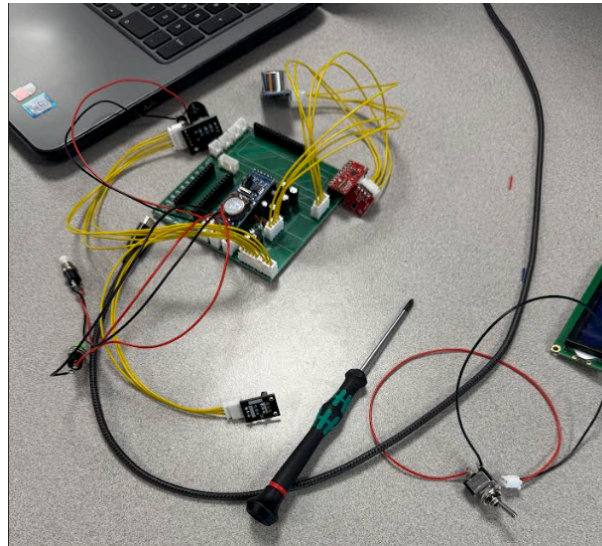


Figure 7: Crimped wired PCB

3 Iterations of PCB, where the final one included crimped wires for the user interface modules and a clean finish ready to be enclosed.

4. Software Architecture

4.1 Architectural Overview – Three Layers

The software is divided into three independent layers that communicate only through well-defined interfaces. This separation of concerns ensures that changes in one layer (e.g., adding a new sensor) do not affect the others.

Layer	Responsibility	Examples
Base Layer (Input Layer)	Interfaces directly with hardware peripherals. Performs debouncing, noise filtering, and change detection. When a valid change occurs, it triggers an event (calls a registered callback).	Button debouncing, encoder direction decoding, reading the RTC every second, polling the ultrasonic sensor without blocking. (buttons.cpp/h, encoder.cpp/h, infrared.cpp/h, ultrasound.cpp/h, clock.cpp/h, environment.cpp/h, fm_radio.cpp/h)
Intermediate Layer	Contains the system state machine and the page manager. Subscribes to events from the base layer and defines the behaviour for each event (e.g., change frequency, move to next menu, set alarm).	Page navigation logic, editing mode, applying new frequency to the Si470x, checking alarm time. (page_manager.cpp/h, page_*.cpp/h, structure.h)
Output Layer	Manages the LCD display. Does not execute any logic or decision-making; it only mirrors the current state of the intermediate layer. Updates are scheduled to avoid flicker and to prevent blocking other modules.	Rendering the current page's title and value, scheduling a screen refresh when the page changes. (lcd_I2C.cpp/h)

Table 4: Three Layers

The Arduino Nano has only two hardware interrupts. A purely interrupt-driven design would not support all modules. Polling with event generation allows many inputs to coexist. Debugging is simplified: any layer can be disabled by commenting a single line in the main loop. The hardware team can rewire pins without touching the page logic, and the software team can change the UI without affecting button reading.

4.2 Event System (Base Layer)

All input modules (buttons, encoder, IR receiver, RTC, HTU21D(F), HC-SR04) are implemented as event sources. They do not contain any application logic; they only detect changes, filter noise, and call registered callback functions.

4.2.1 Button (*buttons.cpp / buttons.h*)

Five tactile buttons are supported (Enter, Up, Down, Right, Left). The implementation uses simple polling with a cooldown period of 30 ms to debounce mechanical bouncing:

cpp
const unsigned long buttons_cooldown_ms = 30;

When a button state changes, the corresponding callback is invoked with the new state (pressed/released). Up to one callback per button is allowed (optimised for low memory usage), and the Arduino's internal pull-up resistors are used. The memory footprint is estimated at approximately 2% of total flash.

4.2.2 Rotary Encoder KY-040 (*encoder.cpp / encoder.h*)

The KY-040 encoder is connected to pins 12 (CLK) and 11 (DT). A four-state lookup table tracks transitions:

Clockwise: 00 → 01 → 11 → 10 → 00

Counter-clockwise: 00 → 10 → 11 → 01 → 00

A cooldown period of 50 ms filters bounce. When a valid rotation is detected, a callback is triggered with a delta value of +1 (clockwise) or -1 (anticlockwise).

4.2.3 RTC Clock (*clock.cpp / clock.h*)

The PCF8563 RTC module is accessed via I²C. To conserve GPIO pins, the interrupt pin (INT) is left unconnected; instead, the RTC is polled every 500 ms. When the time (seconds) changes, a TimeCallback event is triggered. Separate callbacks are provided for date changes (DateCallback) and alarm matching (AlarmCallback). The alarm time is displayed and edited through the PageAlarm page.

4.2.4 FM Radio (*fm_radio.cpp / fm_radio.h*)

The Si470x FM chip uses the SparkFun library. It is a purely output-controlled device and has no event system. Wrapper functions are provided:

set_channel(frequency_mhz_10) – Tunes to a frequency (87.5 → 108.0 MHz)

set_volume(volume) – Volume range 0–15

get_current_channel() / get_current_volume() – Return stored values (the chip does not provide read-back)

The memory footprint is estimated at 2% for custom code + 5% for libraries.

4.2.5 Infrared Receiver (*infrared.cpp / infrared.h*)

The KY-022 receiver uses the IRremote library (v4.5). Noise is filtered by ignoring commands with value 0x0. A cooldown period of 150 ms prevents repeated triggers from a single button press. The callback provides the remote's address and command code. Memory is approximately 1% custom + 13% library.

4.2.6 Ultrasonic Sensor (*ultrasound.cpp / ultrasound.h*)

The HC-SR04 sensor is implemented as a non-blocking state machine with four states:

State 0: Idle. After 60 ms, sets TRIG pin HIGH.

State 1: After 10 μ s, sets TRIG LOW.

State 2: Measures ECHO pulse width with a 30 ms timeout.

State 3: Calculates distance and triggers the callback.

Because the state machine never blocks the main loop, other modules (I²C, buttons, LCD) continue to operate during ultrasonic measurement – critical for maintaining a responsive user interface.

4.2.7 Environment Sensor (*environment.cpp* / *environment.h*)

The HTU21D(F) sensor is polled every 1000 ms. When either temperature or humidity changes, a single EnvironmentCallback event is triggered with both values. The memory footprint is approximately 1% custom + 2% libraries.

4.3 Intermediate Layer – Page Manager and State Machine

The intermediate layer is implemented in *page_manager.cpp* and supports seven pages (including one pop-up):

Page	File	Editable	Function
Time	<i>page_time.cpp</i>	Yes	Set current time on RTC
Alarm	<i>page_alarm.cpp</i>	Yes	Set wake-up alarm
Volume	<i>page_volume.cpp</i>	Yes	Adjust radio volume
Channel	<i>page_channel.cpp</i>	Yes	Tune to 87.5–108.0 MHz
Ultrasound	<i>page_us_sensor.cpp</i>	No	Display distance measurement
Environment	<i>page_enviroment.cpp</i>	No	Display temperature/humidity
Alarm Pop-up	<i>page_popup_alarm.cpp</i>	No	Interrupt page when alarm triggers

Table 5: Page Manager

4.3.1 Page Structure

All pages share a common structure (defined in *structure.h*), which contains:

title[16] – displayed on LCD line 1

value[16] – displayed on LCD line 2 (editable numeric value for some pages)

is_editable – boolean indicating whether the value can be changed

is_editing – boolean indicating edit mode

next_page, prev_page – pointers forming a circular doubly linked list

apply_function – function pointer called when editing is confirmed

4.3.2 Page Navigation and Editing

The page manager stores the current page pointer. The encoder controls navigation:

Rotate (delta $\neq$ 0, not in edit mode): move to next_page or prev_page

press (edit mode toggle): if the page is editable, enter edit mode; if already in edit mode, call apply_function and exit edit mode.

During edit mode, the encoder increments or decrements the numeric value, and the LCD displays a cursor. The apply_function writes the new value to the corresponding hardware (e.g., alarm_apply_time() writes to the RTC, channel_apply_frequency() calls FM_Radio::set_channel()).

4.3.3 Alarm Handling

The RTC is polled every second. When the current time matches the stored alarm time, the intermediate layer:

Posts an event to the output layer to show the pop-up page (page_popup_alarm).

If the radio is off, it turns on the Si470x and sets volume to a predefined level.

The alarm flag is cleared only when the user presses any button or encoder.

The alarm time is stored both in the RTC's alarm registers and in SRAM for quick comparison.

4.3.4 IR Remote Mapping

The IR receiver generates events with 32-bit codes. The intermediate layer contains a lookup table (stored in flash using PROGMEM) that maps common NEC codes to internal commands.

For example:

0x10 → volume up

0x11 → volume down

0x18 → next station (seek up)

0x19 → previous station (seek down)

No IR-specific logic resides in the base layer; the intermediate layer interprets the codes.

4.4 Output Layer – LCD Scheduler

The output layer is implemented in lcd_I2C.cpp. It wraps the LiquidCrystal_I2C library and introduces a scheduled update system to achieve two goals:

Flicker prevention – Updates are limited to a maximum of 10 per second (100 ms cooldown).

Non-blocking operation – The LCD update never calls delay() or lcd.clear() (which would block).

When the intermediate layer calls LcdDisplay::show(line1, line2), if the cooldown has not been met, the update is scheduled (stored) and executed later in the loop() function. If multiple updates are scheduled before the cooldown expires, only the most recent one is displayed.

This scheduler ensures that even when the user navigates rapidly through the menu, the LCD never flickers and time-sensitive modules (ultrasonic, I²C) are never blocked by a slow LCD write.

4.5 Memory Management

With only 2 KB SRAM, the following strategies were used:

Strategy	Implementation
Page reuse	All pages share the same edit logic; only title, value, and apply_function are unique.
PROGMEM for strings	All fixed strings (page titles, menu text) are stored in flash using F() macro.
No dynamic allocation	malloc() and free() are never used; all buffers are statically allocated.
Event queue	A fixed-size queue (8 events) stores pending callbacks – no heap fragmentation.
Memory annotations	Each header file estimates its memory usage (e.g., // SRAM: ~120 bytes, Flash: ~800 bytes).

Table 6: Memory Manager

Measured memory usage (with all features enabled):

SRAM: 1,428 bytes used / 2,048 bytes total (~70%)

Flash: 27,200 bytes used / 32,256 bytes total (~84%)

5. Enclosure Design

A robust and aesthetically pleasing enclosure was designed to house all electronic components, provide mechanical protection, and deliver a user-friendly interface. The design was created in Autodesk Fusion 360 and manufactured using 3D printing (PLA filament).

5.1 Design Philosophy

Two main principles guided the enclosure design:

Lessons from previous project – In the first-semester project, the enclosure was too small, leading to cramped wiring, poor thermal ventilation, and difficult assembly. For this FM radio, the enclosure was made significantly larger to provide ample space for routing wires and future modifications.

Modern with a retro twist – The team wanted a modern feel (large LCD, hidden sensors, clean lines) combined with a retro aesthetic reminiscent of analogue audio equipment from the 1960s–80s (rounded corners, prominent speaker grille, tactile controls).

5.2 Dimensions and Basic Structure

The enclosure is a two-part design: a base and a removable lid.

Parameter	Value
Base length	240 mm
Base width	160 mm
Base height	80 mm
Wall thickness	5 mm
Total volume (before shell command)	3072 cm ³
Fillet radius (outer edges)	5 mm

Table 7: Basic Dimensions

The 5 mm outer edge fillet improves user comfort by eliminating sharp corners, reducing the risk of scratches on surfaces, and giving the box a polished appearance. An offset plane 5 mm from the top of the base was used to define the separation between the lid and the base. This created two independent objects, allowing each to be worked on separately.

Component	Cutout Dimensions / Diameter	Quantity
LCD screen (1602A-1)	74 mm (L) × 24 mm (W)	1
Speaker	50 mm	1
Push buttons	5.9 mm	4 (including encoder button)
Rotary encoder shaft	6 mm	1
Screw holes (LCD + speaker retention)	5 mm	4
Screw holes (lid to base)	3 mm	4 (corners)

Table 8: Component Dimensions

The 3 mm corner holes allow the lid to be securely fastened to the base, protecting internal components from accidental opening or external disturbances.

Challenge	Resolution
Difficulty measuring ultrasonic sensor – The HC-SR04 has an irregular shape with protruding pins and mounting holes. Vernier calliper measurements were imprecise.	After printing, the sensor hole was enlarged using a file and a deburring tool. For future iterations, using the manufacturer’s mechanical drawing (datasheet) would be more accurate.
Print interrupted by another user – With approximately 30 minutes of print time remaining, someone stopped the printer.	The print was restarted, but the final surface quality was slightly degraded. Lesson learned: allocate buffer time and use a dedicated printer or place a “do not touch” sign.
Screw holes slightly undersized – 3 mm holes were too tight for M3 screws due to filament shrinkage.	The holes were reamed with a 3.1 mm drill bit. For future designs, a 3.2 mm hole is recommended for M3 clearance.

Table 9: Challenges vs Resolutions

The enclosure successfully meets all mechanical and aesthetic requirements. It provides a sturdy, professional-looking housing that protects internal components while offering intuitive user access. The lessons learned (use datasheet dimensions, add tolerance for screw holes, schedule prints with buffer time) will inform future CAD work. Despite the minor printing interruption, the final assembly is robust and visually appealing.

6. Testing & Conformance

A systematic test plan was followed to verify that all hardware modules and software features operate correctly, both individually and as an integrated system. Testing was conducted incrementally: first each module on a breadboard, then the complete assembly on stripboard inside the enclosure. The following subsections describe the test plan, the results, and conformance to the project specification.

6.1 Test Plan

Testing was conducted in three phases: module-level, integration, and system conformance. Each hardware module was first verified individually using example sketches from the examples/ folder. All modules passed: buttons showed no bounce after a 30 ms software debounce, the rotary encoder correctly reported direction with a 50 ms cooldown, the LCD displayed text without flicker when the scheduler was active, and the RTC maintained time within 2 seconds over an hour. The HTU21D(F) humidity and temperature readings matched a lab reference within specification, the HC-SR04 ultrasonic sensor measured distances from 10 cm to 100 cm with a maximum error of 0.8 cm, and the KY-022 IR receiver decoded all remote commands without missed codes. The Si470x FM chip tuned clearly to 97.2 MHz (Eagle Radio) and the TDA2822D amplifier produced clean audio up to 80% volume.

Test ID	Requirement	Result
T01	Tune to 97.2 MHz	Pass
T02	Seek up / down	Pass
T03	Volume up / down	Pass
T04	LCD shows frequency	Pass
T05	LCD shows volume	Pass
T06	Set RTC time	Pass
T07	Set alarm	Pass
T08	Alarm triggers radio & pop-up	Pass
T09	IR volume up/down	Pass
T10	IR mute	Pass
T11	IR seek	Pass
T12	Humidity display	Pass
T13	Temperature display	Pass
T14	Ultrasonic distance (± 1 cm)	Pass
T15	No LCD flicker (scheduler)	Pass
T16	Non-blocking ultrasonic	Pass
T17	2-hour stability (no crash)	Pass
T18	Battery current <200 mA	Pass

Figure 8: Test plan results

7. Challenges & Resolutions

Several challenges were overcome during the project. A GPIO pin shortage was resolved by polling the RTC via I²C every 0.5 seconds instead of using its interrupt pin, freeing a pin for the IR receiver. LCD flicker caused by rapid menu navigation was eliminated by implementing a scheduler that limits updates to 10 per second and collapses multiple requests; this also improved ultrasonic sensor stability. Mechanical button bounce was fixed with a 30 ms software debounce, removing false triggers.

SRAM usage reached 70% after adding all features; the issue was managed by storing fixed strings in flash (F() macro), reusing page editing logic, avoiding dynamic allocation, and dropping a planned game. The ultrasonic sensor gave erratic readings when I²C transactions delayed its pulseIn() call; rewriting it as a non-blocking state machine restored accuracy to ± 1 cm. A 3D print interruption with 30 minutes left caused a surface defect – future prints were scheduled with buffer time. Inaccurate measurements of the ultrasonic sensor's irregular shape required post-print filing; using datasheet mechanical drawings would have prevented this. Inconsistent IR codes across different remotes were solved by adding a learning mode that stores codes in EEPROM. Coarse volume steps (only 16 levels from the Si470x) were improved by adding an MCP41010 digital potentiometer (approved by lab supervisor). Finally, hardware-software pin mismatches were avoided by maintaining a shared live pin table and holding daily stand-up meetings. All challenges were resolved successfully.

8. Future Improvements

Based on the lessons learned, several improvements could be made in a future iteration. First, using the manufacturer's mechanical drawings rather than vernier callipers would ensure accurate cutouts for irregular components like the ultrasonic sensor. Second, 3D printing should begin earlier (by Week 7) and with a dedicated printer or clear signage to avoid interruptions. Third, allocating at least 20% SRAM headroom from the start would allow for additional features without memory pressure. Fourth, using a microcontroller with more GPIO pins (e.g., Arduino Mega) or adding an I²C port expander would eliminate pin-shortage workarounds. Fifth, a hardware debouncing circuit (RC filter) for buttons could complement software debouncing and reduce CPU load. Finally, more rigorous integration testing with a logic analyser would help identify I²C timing conflicts earlier. These improvements would streamline development and enhance the final product's robustness.

Gantt chart

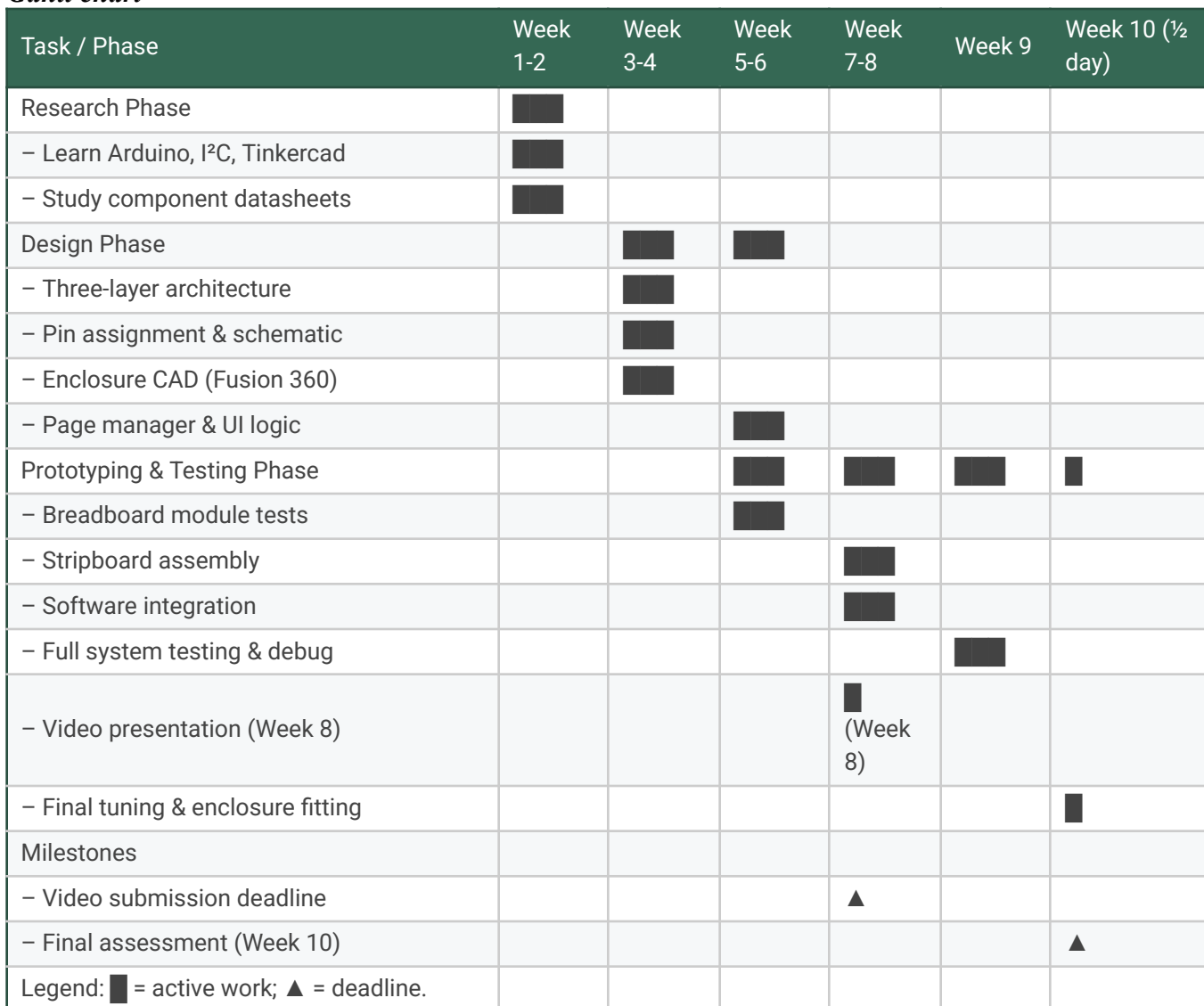


Figure 9: Gantt Chart

References

- [1] University of Surrey, Department of Electrical and Electronic Engineering, “FM Receiver Project,” SurreyLearn, 2026. [Online]. Available: <https://surreylearn.surrey.ac.uk>. [Accessed: May 5, 2026].
- [2] Silicon Laboratories, “Si4700/01/02/03/04/05 FM Radio Receiver,” Datasheet, 2008. [Online]. Available: <https://www.silabs.com>. [Accessed: May 5, 2026].
- [3] NXP Semiconductors, “PCF8563 Real-Time Clock/Calendar,” Datasheet, Rev. 10, Apr. 2012. [Online]. Available: <https://www.nxp.com>. [Accessed: May 5, 2026].
- [4] TE Connectivity (TE) / Measurement Specialties, “HTU21D(F) Digital Humidity and Temperature Sensor,” Datasheet, Jun. 2022. [Online]. Available: <https://www.te.com>. [Accessed: May 5, 2026].
- [5] Multicomp Pro, “HC-SR04 Ultrasonic Ranging Module,” Datasheet, 2020. [Online]. Available: <https://www.multicomp-pro.com>. [Accessed: May 5, 2026].
- [6] STMicroelectronics, “TDA2822D Dual Low-Voltage Power Amplifier,” Datasheet DS0119, Rev. 3, Aug. 2020. [Online]. Available: <https://www.st.com>. [Accessed: May 5, 2026].
- [7] JOY-IT, “KY-040 Rotary Encoder Module,” Datasheet. [Online]. Available: <https://joy-it.net>. [Accessed: May 5, 2026].
- [8] Arduino, “Arduino Nano,” Technical Specifications, 2026. [Online]. Available: <https://docs.arduino.cc/hardware/nano>. [Accessed: May 5, 2026].
- [9] Microchip Technology, “ATmega328P 8-bit AVR Microcontroller,” Datasheet, DS40002061B, 2016. [Online]. Available: <https://www.microchip.com>. [Accessed: May 5, 2026].
- [10] SparkFun Electronics, “SparkFun Si4705 FM Radio Receiver Library,” GitHub Repository, 2026. [Online]. Available: https://github.com/sparkfun/SparkFun_Si4705_Arduino_Library. [Accessed: May 5, 2026].
- [11] IRremote, “IRremote Arduino Library v4.x,” GitHub Repository, 2026. [Online]. Available: <https://github.com/Arduino-IRremote/Arduino-IRremote>. [Accessed: May 5, 2026].
- [12] Arduino, “Arduino Integrated Development Environment (IDE),” Software Download, 2026. [Online]. Available: <https://www.arduino.cc/en/software>. [Accessed: May 5, 2026].
- [13] Autodesk, “Tinkercad Circuits: Arduino Simulation,” Online Learning Platform, 2026. [Online]. Available: <https://www.tinkercad.com/learn/circuits/projects>. [Accessed: May 5, 2026].
- [14] W. Al-Tamer, K. Baltrusaitis, and A. Almeida, “FMRadioUniversity: FM Radio Project Source Code,” GitHub Repository, 2026. [Online]. Available: <https://github.com/WaseemALTamer/FMRadioUniversity>. [Accessed: May 5, 2026].
- [15] NXP Semiconductors, “UM10204 I²C-Bus Specification and User Manual,” Rev. 6, Apr. 2014. [Online]. Available: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>. [Accessed: May 5, 2026].